

Week 6: Text Mining

PM 566: Introduction to Health Data Science

Emil Hvitfeldt, George G. Vega Yon, Kelly Street

Acknowledgment

These slides were originally developed by Emil Hvitfeldt and modified by George G. Vega Yon and Kelly Street.

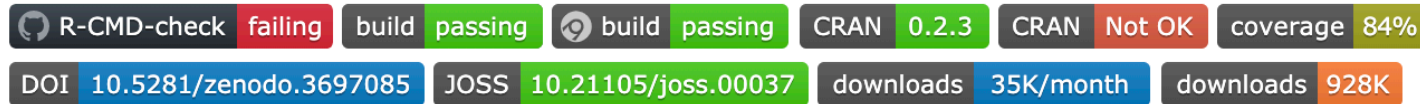
Plan for the week

- We will try to turn text into numbers
- Then use tidy principals to explore those numbers

tidytext: Text mining using dplyr, ggplot2, and other tidy tools

Authors: [Julia Silge](#), [David Robinson](#)

License: [MIT](#)



Using [tidy data principles](#) can make many text mining tasks easier, more effective, and consistent with tools already in wide use. Much of the infrastructure needed for text mining with tidy data frames already exists in packages like [dplyr](#), [broom](#), [tidyr](#) and [ggplot2](#). In this package, we provide functions and supporting data sets to allow conversion of text to and from tidy formats, and to switch seamlessly between tidy tools and existing text mining packages. Check out [our book](#) to learn more about text mining using tidy data principles.

Why tidytext?

Works seamlessly with ggplot2, dplyr and tidyr.

Alternatives:

R: quanteda, tm, koRpus

Python: nltk, Spacy, gensim

Alice's Adventures in Wonderland

Download the `alice` dataset from

<https://github.com/USCbiostats/PM566/blob/main/data/alice.rds>)

```
1 library(tidyverse)
2 alice <- readRDS("../data/alice.rds") # "~/Downloads/alice.rds"
3 alice
```

```
# A tibble: 3,351 × 3
```

text	chapter
chapter_name	
<chr>	<int> <chr>
1 "CHAPTER I."	1 CHAPTER I.
2 "Down the Rabbit-Hole"	1 CHAPTER I.
3 ""	1 CHAPTER I.
4 ""	1 CHAPTER I.
5 "Alice was beginning to get very tired of sitting by he...	1 CHAPTER I.
6 "bank, and of having nothing to do: once or twice she h...	1 CHAPTER I.
7 "the book her sister was reading, but it had no picture...	1 CHAPTER I.
8 "conversations in it, "and what is the use of a book," ...	1 CHAPTER I.
9 ""without pictures or conversations?"	1 CHAPTER I.
10 ""	1 CHAPTER I.

```
# i 3,341 more rows
```


Tokenizing

Turning text into smaller units

In English:

- split by spaces
- more advanced algorithms

Spacy tokenizer

1. Iterate over whitespace-separated substrings.
2. Look for a token match. If there is a match, stop processing and keep this token.
3. Check whether we have an explicitly defined special case for this substring. If we do, use it.
4. Otherwise, try to consume one prefix. If we consumed a prefix, go back to #2, so that the token match and special cases always get priority.
5. If we didn't consume a prefix, try to consume a suffix and then go back to #2.
6. If we can't consume a prefix or a suffix, look for a URL match.
7. If there's no URL match, then look for a special case.
8. Look for "infixes" — stuff like hyphens etc. and split the substring into tokens on all infixes.
9. Once we can't consume any more of the string, handle it as a single token.

Turning the data into a tidy format

```
1 library(tidytext)
2 alice |>
3   unnest_tokens(token, text)
```

```
# A tibble: 26,687 × 3
```

	chapter	chapter_name	token
	<int>	<chr>	<chr>
1	1	CHAPTER I.	chapter
2	1	CHAPTER I.	i
3	1	CHAPTER I.	down
4	1	CHAPTER I.	the
5	1	CHAPTER I.	rabbit
6	1	CHAPTER I.	hole
7	1	CHAPTER I.	alice
8	1	CHAPTER I.	was
9	1	CHAPTER I.	beginning
10	1	CHAPTER I.	to

```
# i 26,677 more rows
```


Words as a unit

Now that we have words as the observation unit we can use the **dplyr** toolbox.

Using **dplyr** verbs

```
1 library(dplyr)
2 alice |>
3   unnest_tokens(token, text)
```

```
# A tibble: 26,687 × 3
```

	chapter	chapter_name	token
	<int>	<chr>	<chr>
1	1	CHAPTER I.	chapter
2	1	CHAPTER I.	i
3	1	CHAPTER I.	down
4	1	CHAPTER I.	the
5	1	CHAPTER I.	rabbit
6	1	CHAPTER I.	hole
7	1	CHAPTER I.	alice
8	1	CHAPTER I.	was
9	1	CHAPTER I.	beginning
10	1	CHAPTER I.	to

```
# i 26,677 more rows
```

Using **dplyr** verbs

```
1 library(dplyr)
2 alice |>
3   unnest_tokens(token, text) |>
4   count(token)
```

```
# A tibble: 2,740 × 2
```

	token	n
	<chr>	<int>
1	_alice's	1
2	_all	1
3	_all_	1
4	_and	1
5	_are_	4
6	_at	1
7	_before	1
8	_beg_	1
9	_began_	1
10	_best_	2

```
# i 2,730 more rows
```

Using **dplyr** verbs

```
1 library(dplyr)
2 alice |>
3   unnest_tokens(token, text) |>
4   count(token, sort = TRUE)
```

```
# A tibble: 2,740 × 2
```

	token	n
	<chr>	<int>
1	the	1643
2	and	871
3	to	729
4	a	632
5	she	538
6	it	527
7	of	514
8	said	460
9	i	393
10	alice	386

```
# i 2,730 more rows
```

Using **dplyr** verbs

```
1 library(dplyr)
2 alice |>
3   unnest_tokens(token, text) |>
4   count(chapter, token)
```

A tibble: 7,549 × 3

	chapter	token	n
	<int>	<chr>	<int>
1	1	_curtseying_	1
2	1	_never_	1
3	1	_not_	1
4	1	_one_	1
5	1	_poison_	1
6	1	_that_	1
7	1	_through_	1
8	1	_took	1
9	1	_very_	4
10	1	_was_	1

i 7,539 more rows

Using **dplyr** verbs

```
1 library(dplyr)
2 alice |>
3   unnest_tokens(token, text) |>
4   group_by(chapter) |>
5   count(token) |>
6   top_n(10, n)
```

```
# A tibble: 122 × 3
```

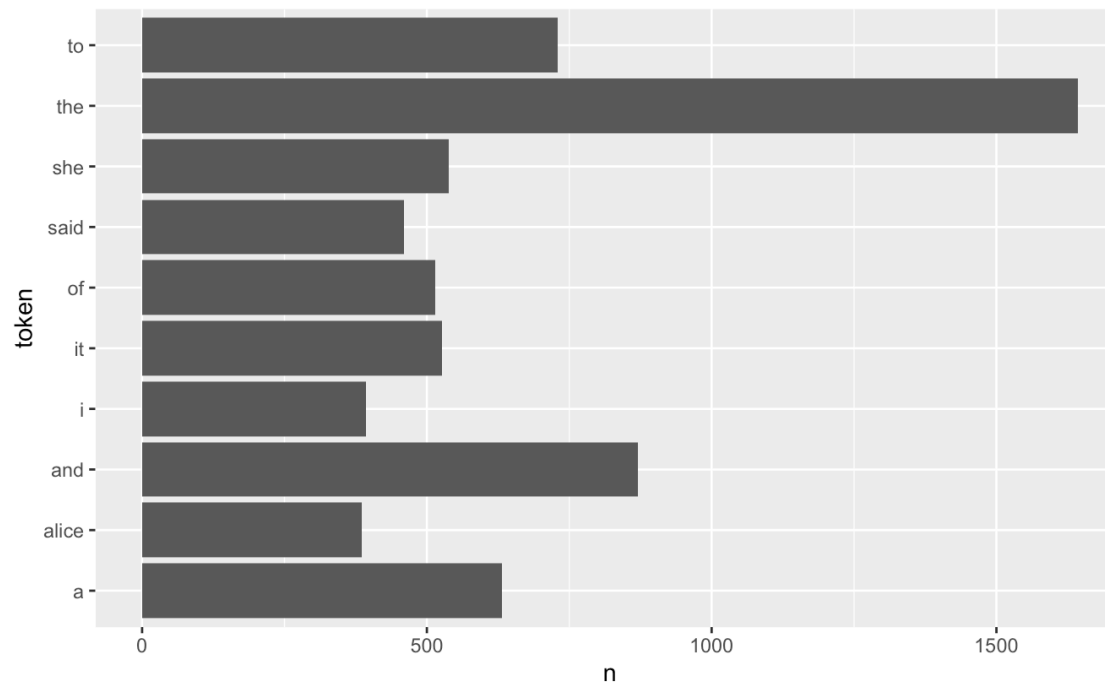
```
# Groups:   chapter [12]
```

	chapter	token	n
	<int>	<chr>	<int>
1	1	a	52
2	1	alice	27
3	1	and	65
4	1	i	30
5	1	it	62
6	1	of	43
7	1	she	79
8	1	the	92
9	1	to	75
10	1	was	52

```
# i 112 more rows
```

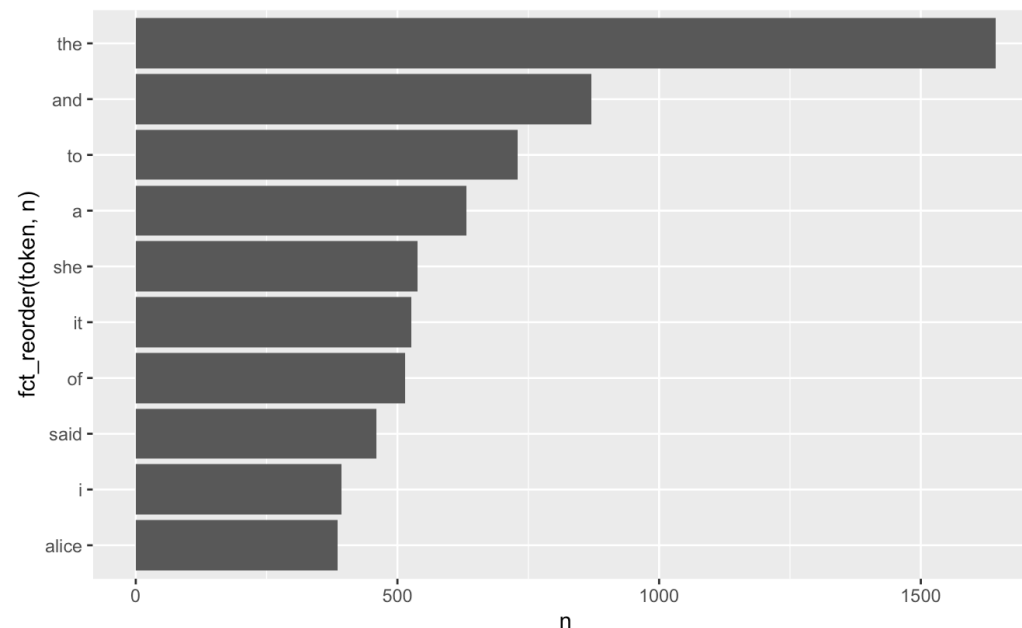
dplyr verbs and ggplot2

```
1 library(dplyr)
2 library(ggplot2)
3 alice |>
4   unnest_tokens(token, text) |>
5   count(token) |>
6   top_n(10, n) |>
7   ggplot(aes(n, token)) +
8   geom_col()
```



dplyr verbs and ggplot2

```
1 library(dplyr)
2 library(ggplot2)
3 library(forcats)
4 alice |>
5   unnest_tokens(token, text) |>
6   count(token) |>
7   top_n(10, n) |>
8   ggplot(aes(n, fct_reorder(token, n))) +
9   geom_col()
```



Stop words

A lot of the words don't tell us very much. Words such as “the”, “and”, “at” and “for” appear a lot in English text but doesn't add much to the context.

Words such as these are called **stop words**

For more information about differences in stop words and when to remove them read this chapter:

<https://smltar.com/stopwords>

Stop words in tidytext

`tidytext` comes with a built-in `data.frame` of stop words

```
1 stop_words
```

```
# A tibble: 1,149 × 2
  word      lexicon
  <chr>    <chr>
1 a       SMART
2 a's     SMART
3 able    SMART
4 about   SMART
5 above   SMART
6 according SMART
7 accordingly SMART
8 across  SMART
9 actually SMART
10 after   SMART
# i 1,139 more rows
```

Stop word lexicons

```
1 table(stop_words$lexicon)
```

onix	SMART	snowball
404	571	174

snowball stopwords

```
1 stop_words |>
2   filter(lexicon == "snowball") |>
3   pull(word)
```

[1]	"i"	"me"	"my"	"myself"	"we"
[6]	"our"	"ours"	"ourselves"	"you"	"your"
[11]	"yours"	"yourself"	"yourselves"	"he"	"him"
[16]	"his"	"himself"	"she"	"her"	"hers"
[21]	"herself"	"it"	"its"	"itself"	"they"
[26]	"them"	"their"	"theirs"	"themselves"	"what"
[31]	"which"	"who"	"whom"	"this"	"that"
[36]	"these"	"those"	"am"	"is"	"are"
[41]	"was"	"were"	"be"	"been"	"being"
[46]	"have"	"has"	"had"	"having"	"do"
[51]	"does"	"did"	"doing"	"would"	"should"
[56]	"could"	"ought"	"i'm"	"you're"	"he's"
[61]	"she's"	"it's"	"we're"	"they're"	"i've"
[66]	"you've"	"we've"	"they've"	"i'd"	"you'd"
[71]	"he'd"	"she'd"	"we'd"	"they'd"	"i'll"
[76]	"..."	"..."	"..."	"..."	"..."

Duplicated stopwords

```
1 sort(table(stop_words$word), decreasing = TRUE)
```

down	would	a	about	above
4	4	3	3	3
after	again	against	all	an
3	3	3	3	3
and	any	are	as	at
3	3	3	3	3
be	because	been	before	being
3	3	3	3	3
between	both	but	by	cannot
3	3	3	3	3
could	did	do	does	during
3	3	3	3	3
each	few	for	from	further
3	3	3	3	3
,	,	,	,	,

Removing stopwords

We can use an `anti_join()` to remove the tokens that also appear in the `stop_words` data.frame

```
1 alice |>
2   unnest_tokens(token, text) |>
3   anti_join(stop_words, by = c("token" = "word")) |>
4   count(token, sort = TRUE)
```

```
# A tibble: 2,314 × 2
```

	token	n
	<chr>	<int>
1	alice	386
2	time	71
3	queen	68
4	king	61
5	don't	60
6	it's	57
7	i'm	56
8	mock	56
9	turtle	56
10	gryphon	55

```
# i 2,304 more rows
```

Anti-join with same variable name

```
1  alice |>
2    unnest_tokens(word, text) |>
3    anti_join(stop_words, by = c("word")) |>
4    count(word, sort = TRUE)
```

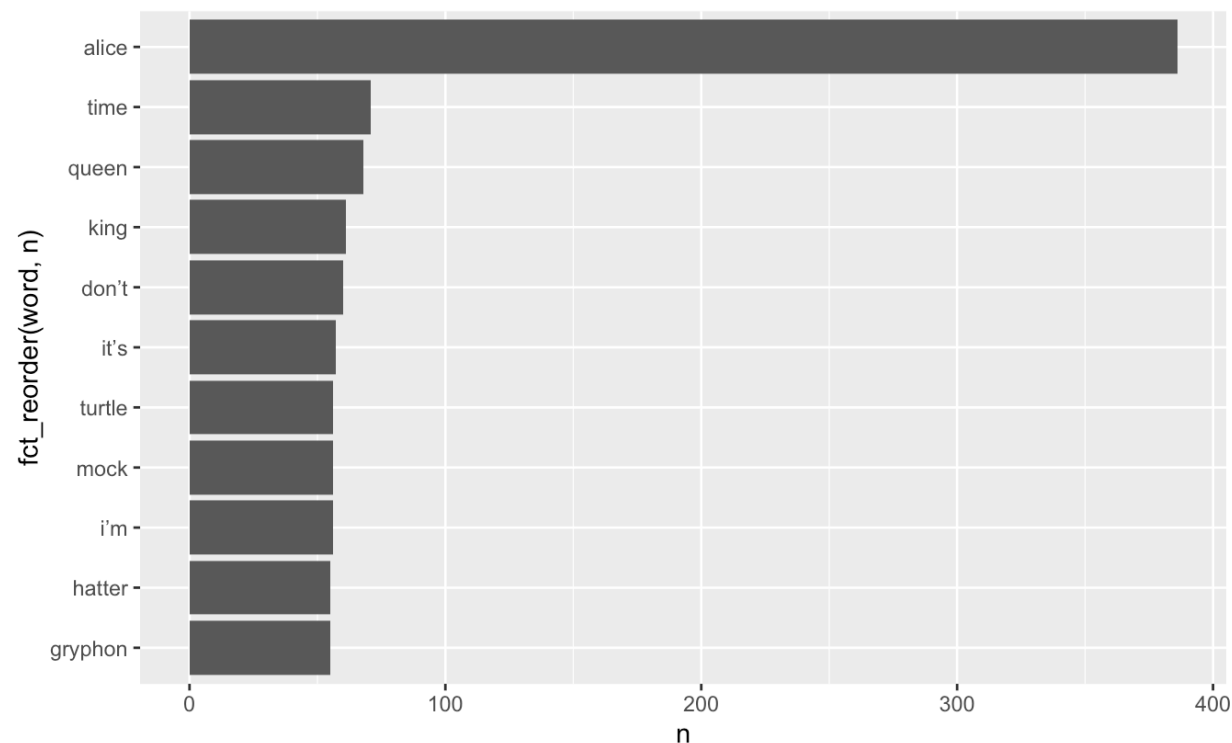
```
# A tibble: 2,314 × 2
```

	word	n
	<chr>	<int>
1	alice	386
2	time	71
3	queen	68
4	king	61
5	don't	60
6	it's	57
7	i'm	56
8	mock	56
9	turtle	56
10	gryphon	55

```
# i 2,304 more rows
```

Stop words removed

```
1 alice |>  
2   unnest_tokens(word, text) |>  
3   anti_join(stop_words, by = c("word")) |>  
4   count(word, sort = TRUE) |>  
5   top_n(10, n) |>  
6   ggplot(aes(n, fct_reorder(word, n))) +  
7   geom_col()
```



Which words appear together?

ngrams are sets of n consecutive words and we can count these to see which words appear together most frequently.

- ngrams with $n = 1$ are called “unigrams”: “which”, “words”, “appear”, “together”
- ngrams with $n = 2$ are called “bigrams”: “which words”, “words appear”, “appear together”
- ngrams with $n = 3$ are called “trigrams”: “which words appear”, “words appear together”

Which words appear together?

We can extract bigrams using `unnest_ngrams()` with `n = 2`

```
1 alice |>  
2   unnest_ngrams(ngram, text, n = 2)
```

```
# A tibble: 25,170 × 3
```

	chapter	chapter_name	ngram
	<int>	<chr>	<chr>
1	1	CHAPTER I.	chapter i
2	1	CHAPTER I.	down the
3	1	CHAPTER I.	the rabbit
4	1	CHAPTER I.	rabbit hole
5	1	CHAPTER I.	<NA>
6	1	CHAPTER I.	<NA>
7	1	CHAPTER I.	alice was
8	1	CHAPTER I.	was beginning
9	1	CHAPTER I.	beginning to
10	1	CHAPTER I.	to get

```
# i 25,160 more rows
```

Which words appear together?

Tallying up the bigrams still shows a lot of stop words, but it is able to pick up some common phrases:

```
1  alice |>
2    unnest_ngrams(ngram, text, n = 2) |>
3    count(ngram, sort = TRUE)
```

```
# A tibble: 13,424 × 2
```

	ngram	n
	<chr>	<int>
1	<NA>	951
2	said the	206
3	of the	130
4	said alice	112
5	in a	96
6	and the	75
7	in the	75
8	it was	72
9	to the	68
10	the queen	60

```
# i 13,414 more rows
```

Which words appear together?

```
1  alice |>
2    unnest_ngrams(ngram, text, n = 2) |>
3    separate(ngram, into = c("word1", "word2"), sep = " ") |>
4    select(word1, word2)
```

```
# A tibble: 25,170 × 2
```

	word1	word2
	<chr>	<chr>
1	chapter	i
2	down	the
3	the	rabbit
4	rabbit	hole
5	<NA>	<NA>
6	<NA>	<NA>
7	alice	was
8	was	beginning
9	beginning	to
10	to	get

```
# i 25,160 more rows
```

```
1  alice |>
2    unnest_ngrams(ngram, text, n = 2) |>
3    separate(ngram, into = c("word1", "word2"), sep = " ") |>
4    select(word1, word2) |>
5    filter(word1 == "alice")
```

```
# A tibble: 336 × 2
```

```
  word1 word2
  <chr> <chr>
```

```
1  alice was
2  alice think
3  alice started
4  alice after
5  alice had
6  alice to
7  alice had
8  alice had
9  alice soon
10 alice began
# i 326 more rows
```

What about when the first word is “alice”?

```
1  alice |>
2    unnest_ngrams(ngram, text, n = 2) |>
3    separate(ngram, into = c("word1", "word2"), sep = " ") |>
4    select(word1, word2) |>
5    filter(word1 == "alice") |>
6    count(word2, sort = TRUE)
```

A tibble: 133 × 2

	word2	n
	<chr>	<int>
1	and	18
2	was	17
3	thought	12
4	as	11
5	said	11
6	could	10
7	had	10
8	did	9
9	in	9
10	to	9

i 123 more rows

What about when the second word is “alice”?

```
1 alice |>
2   unnest_ngrams(ngram, text, n = 2) |>
3   separate(ngram, into = c("word1", "word2"), sep = " ") |>
4   select(word1, word2) |>
5   filter(word2 == "alice") |>
6   count(word1, sort = TRUE)
```

A tibble: 106 × 2

	word1	n
	<chr>	<int>
1	said	112
2	thought	25
3	to	22
4	and	15
5	poor	11
6	cried	7
7	at	6
8	so	6
9	that	5
10	exclaimed	3

i 96 more rows

TF-IDF

TF: Term frequency

IDF: Inverse document frequency

TF-IDF: product of TF and IDF

TF gives weight to terms that appear a lot, IDF gives weight to terms that appears in a few documents

TF-IDF with `tidytext`

```
1 alice |>  
2   unnest_tokens(text, text)
```

```
# A tibble: 26,687 × 3  
  text      chapter chapter_name  
  <chr>    <int>    <chr>  
1 chapter      1 CHAPTER I.  
2 i            1 CHAPTER I.  
3 down        1 CHAPTER I.  
4 the         1 CHAPTER I.  
5 rabbit      1 CHAPTER I.  
6 hole        1 CHAPTER I.  
7 alice       1 CHAPTER I.  
8 was         1 CHAPTER I.  
9 beginning   1 CHAPTER I.  
10 to         1 CHAPTER I.  
# i 26,677 more rows
```


TF-IDF with `tidytext`

```
1 alice |>  
2   unnest_tokens(text, text) |>  
3   count(text, chapter)
```

A tibble: 7,549 × 3

	text	chapter	n
	<chr>	<int>	<int>
1	_alice's	2	1
2	_all	12	1
3	_all_	12	1
4	_and	9	1
5	_are_	4	1
6	_are_	6	1
7	_are_	8	1
8	_are_	9	1
9	_at	9	1
10	_before	12	1

i 7,539 more rows

TF-IDF with `tidytext`

```
1 alice |>  
2   unnest_tokens(text, text) |>  
3   count(text, chapter) |>  
4   bind_tf_idf(text, chapter, n)
```

A tibble: 7,549 × 6

	text	chapter	n	tf	idf	tf_idf
	<chr>	<int>	<int>	<dbl>	<dbl>	<dbl>
1	_alice's	2	1	0.000471	2.48	0.00117
2	_all	12	1	0.000468	2.48	0.00116
3	_all_	12	1	0.000468	2.48	0.00116
4	_and	9	1	0.000435	2.48	0.00108
5	_are_	4	1	0.000375	1.10	0.000411
6	_are_	6	1	0.000382	1.10	0.000420
7	_are_	8	1	0.000400	1.10	0.000439
8	_are_	9	1	0.000435	1.10	0.000478
9	_at	9	1	0.000435	2.48	0.00108
10	_before	12	1	0.000468	2.48	0.00116

i 7,539 more rows

TF-IDF with `tidytext`

```
1 alice |>  
2   unnest_tokens(text, text) |>  
3   count(text, chapter) |>  
4   bind_tf_idf(text, chapter, n) |>  
5   arrange(desc(tf_idf))
```

A tibble: 7,549 × 6

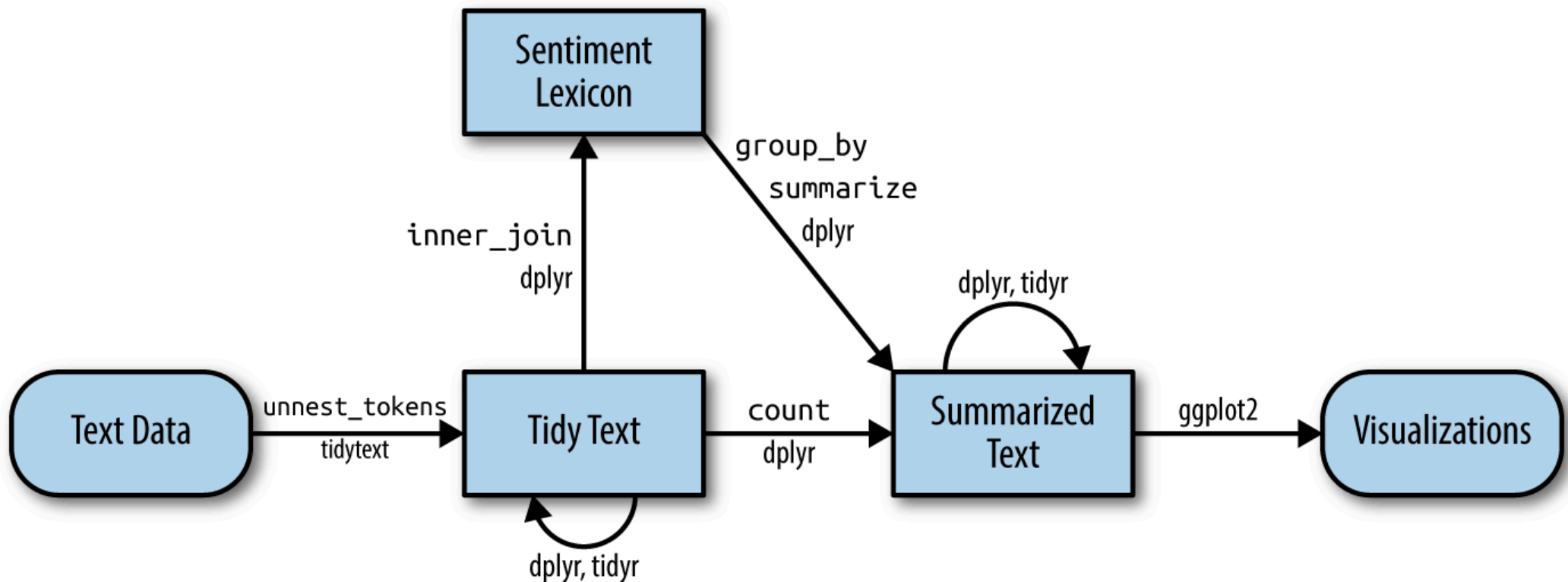
	text	chapter	n	tf	idf	tf_idf
	<chr>	<int>	<int>	<dbl>	<dbl>	<dbl>
1	dormouse	7	26	0.0112	1.79	0.0201
2	hatter	7	32	0.0138	1.39	0.0191
3	mock	10	28	0.0136	1.39	0.0189
4	turtle	10	28	0.0136	1.39	0.0189
5	gryphon	10	31	0.0151	1.10	0.0166
6	turtle	9	27	0.0117	1.39	0.0163
7	caterpillar	5	25	0.0115	1.39	0.0159
8	dance	10	13	0.00632	2.48	0.0157
9	mock	9	26	0.0113	1.39	0.0157
10	hatter	11	21	0.0110	1.39	0.0153

i 7,539 more rows

Sentiment Analysis

Also known as “opinion mining,” sentiment analysis is a way in which we can use computers to attempt to understand the feelings conveyed by a piece of text. This generally relies on a large, human-compiled database of words with known associations such as “positive” and “negative” or specific feelings like “joy”, “surprise”, “disgust”, etc.

Sentiment Analysis



Sentiment Lexicons

The `tidytext` and `textdata` packages provide access to three different databases of words and their associated sentiments (known as “sentiment lexicons”). Obviously, none of these can be perfect, as there is no “correct” way to quantify feelings, but they all attempt to capture different elements of how a text makes you feel.

The readily available lexicons are:

- `afinn` from Finn Årup Nielsen
- `bing` from Bing Liu and collaborators
- `nrc` from Saif Mohammad and Peter Turney

Sentiment Lexicons - **bing**

The **bing** lexicon contains a large list of words and a binary association, either “positive” or “negative”:

```
1 library(textdata)
2 get_sentiments('bing')
```

```
# A tibble: 6,786 × 2
  word      sentiment
  <chr>    <chr>
1 2-faces  negative
2 abnormal negative
3 abolish negative
4 abominable negative
5 abominably negative
6 abominate negative
7 abomination negative
8 abort    negative
9 aborted  negative
10 aborts   negative
# i 6,776 more rows
```

Sentiment Lexicons - **afinn**

The **afinn** lexicon goes slightly further, assigning words a value between -5 and 5 that represents their positivity or negativity.

```
1 get_sentiments('afinn')
```

```
# A tibble: 2,477 × 2
```

	word	value
	<chr>	<dbl>
1	abandon	-2
2	abandoned	-2
3	abandons	-2
4	abducted	-2
5	abduction	-2
6	abductions	-2
7	abhor	-3
8	abhorred	-3
9	abhorrent	-3
10	abhors	-3

```
# i 2,467 more rows
```


Sentiment Lexicons - **nrc**

The **nrc** lexicon takes a different approach and assigns each word an associated sentiment. Some words appear more than once because they have multiple associations:

```
1 get_sentiments('nrc')
```

```
# A tibble: 13,872 × 2
  word      sentiment
<chr>    <chr>
1 abacus    trust
2 abandon   fear
3 abandon   negative
4 abandon   sadness
5 abandoned anger
6 abandoned fear
7 abandoned negative
8 abandoned sadness
9 abandonment anger
10 abandonment fear
# i 13,862 more rows
```

Sentiment Analysis

We can use one of these databases to analyze *Alice's Adventures in Wonderland* by breaking the text down into words and combining the result with a lexicon. Let's use [bing](#) to assign “positive” and “negative” labels to as many words as possible in the book. (Note that this time the variable created by [unnest_tokens](#) is called [word](#), to match the column name in [bing](#)).

```
1 alice |>
2   unnest_tokens(word, text) |>
3   inner_join(get_sentiments("bing"))
```

```
# A tibble: 1,409 × 4
  chapter chapter_name word      sentiment
  <int>   <chr>      <chr>      <chr>
1       1 CHAPTER I.   tired     negative
2       1 CHAPTER I.   well      positive
3       1 CHAPTER I.   hot       positive
4       1 CHAPTER I.   stupid    negative
5       1 CHAPTER I.   pleasure  positive
6       1 CHAPTER I.   worth     positive
7       1 CHAPTER I.   trouble   negative
8       1 CHAPTER I.   remarkable positive
9       1 CHAPTER I.   burning   negative
10      1 CHAPTER I.   fortunately positive
# i 1,399 more rows
```

Sentiment Analysis

We can now group and summarize this new dataset the same as any other. For example, let's look at the sentiment by chapter. We'll do this by counting the number of “positive” words and subtracting the number of “negative” words:

```
1 diff_by_chap <- alice |>
2   unnest_tokens(word, text) |>
3   inner_join(get_sentiments("bing")) |>
4   group_by(chapter) |>
5   summarise(sentiment = sum(sentiment == "positive") - sum(sentiment == "negative"))
```

Sentiment Analysis

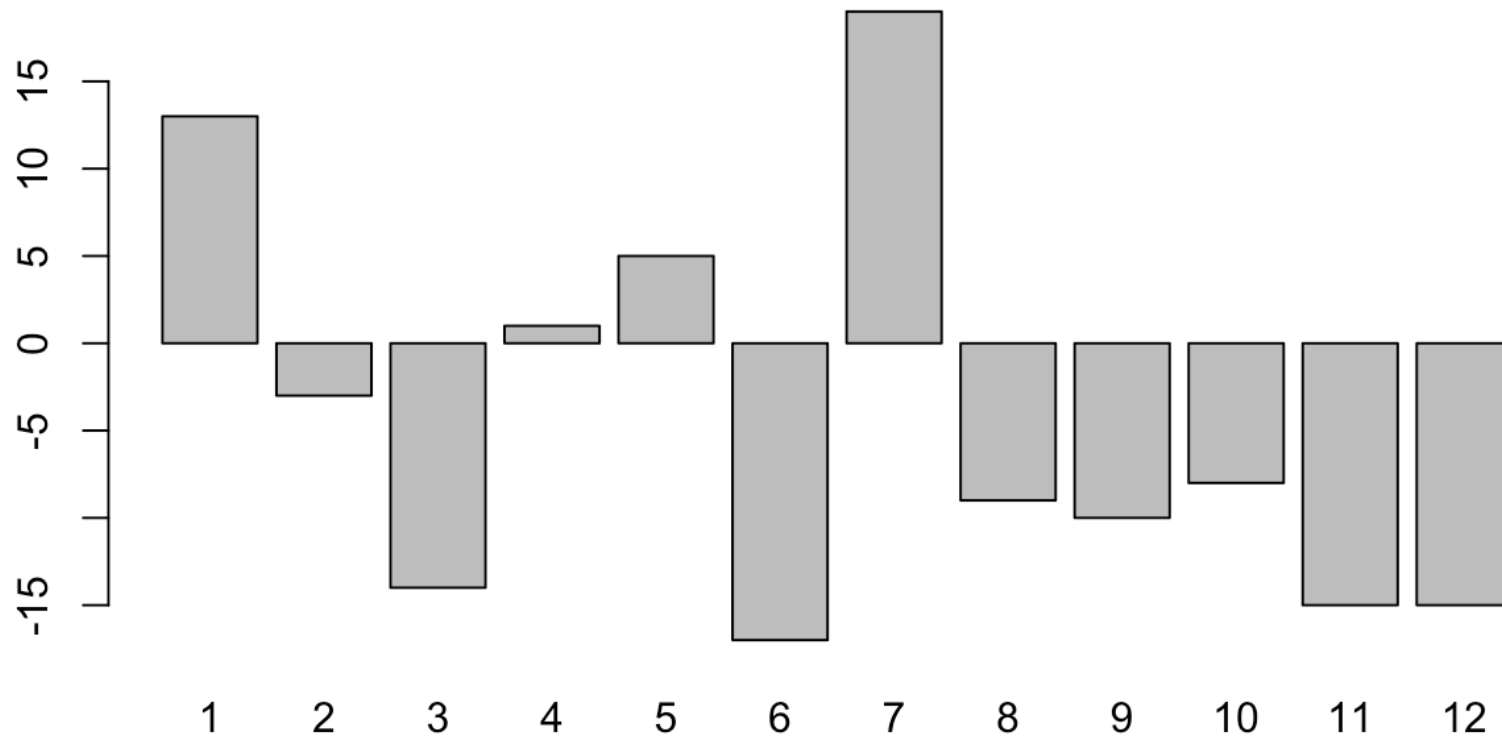
```
1 diff_by_chap
```

```
# A tibble: 12 × 2
```

	chapter	sentiment
	<int>	<int>
1	1	13
2	2	-3
3	3	-14
4	4	1
5	5	5
6	6	-17
7	7	19
8	8	-9
9	9	-10
10	10	-8
11	11	-15
12	12	-15

Sentiment Analysis

```
1 barplot(diff_by_chap$sentiment, names.arg = diff_by_chap$chapter)
```



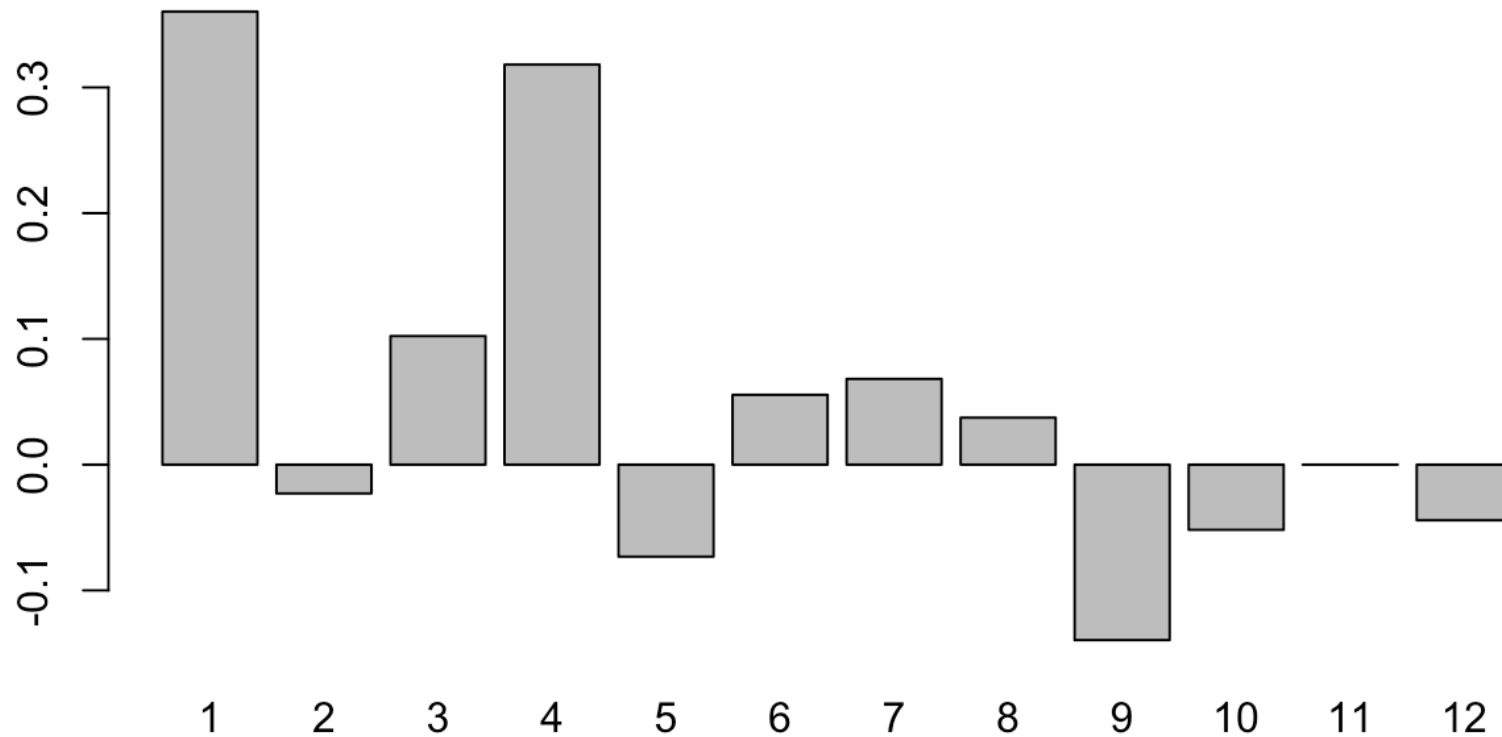
Sentiment Analysis

Alternatively, we could use the `afinn` lexicon and quantify the “sentiment” of each chapter by the average of all words with numeric associations:

```
1 avg_by_chap <- alice |>
2   unnest_tokens(word, text) |>
3   inner_join(get_sentiments("afinn")) |>
4   group_by(chapter) |>
5   summarise(sentiment = mean(value))
```

Sentiment Analysis

```
1 barplot(avg_by_chap$sentiment, names.arg = avg_by_chap$chapter)
```



Sentiment Analysis

Similarly, we can find the most frequent sentiment association in the [nrc](#) lexicon for each chapter. Unfortunately, for all chapters, the most frequent sentiment association ends up being the rather bland “positive” or “negative”:

```
1  alice |>
2    unnest_tokens(word, text) |>
3    inner_join(get_sentiments("nrc")) |>
4    group_by(chapter) |>
5    summarise(sentiment = names(which.max(table(sentiment))))
```

```
# A tibble: 12 × 2
  chapter sentiment
  <int> <chr>
1       1 positive
2       2 positive
3       3 positive
4       4 positive
5       5 positive
6       6 negative
7       7 positive
8       8 positive
9       9 positive
10      10 positive
11      11 positive
12      12 positive
```


Sentiment Analysis

We'll try to spice things up by removing “positive” and “negative” from the **nrc** lexicon:

```
1 nrc_fun <- get_sentiments("nrc")  
2 nrc_fun <- nrc_fun[!nrc_fun$sentiment %in% c("positive", "negative")]
```

Sentiment Analysis

Now we see a lot of “anticipation”:

```
1  alice |>
2    unnest_tokens(word, text) |>
3    inner_join(nrc_fun) |>
4    group_by(chapter) |>
5    summarise(sentiment = names(which.max(table(sentiment))))
```

```
# A tibble: 12 × 2
```

	chapter	sentiment
	<int>	<chr>
1	1	anticipation
2	2	anticipation
3	3	sadness
4	4	anticipation
5	5	trust
6	6	anticipation
7	7	anticipation
8	8	anticipation
9	9	trust
10	10	joy
11	11	anticipation
12	12	trust